

Naive Bayes Multiclass Document Classification

From Word Counts to Probabilistic Classification

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

1. Document Classification

Document Classification

Document classification assigns one or more predefined labels to a document based on its textual content.

Examples for Arabic news:

- Sports
- Politics
- Economy
- Technology
- Culture
- International

Goal

Given a document d , predict its class:

$$\hat{c} = \arg \max_{c \in C} P(c \mid d)$$

2. From Text to Naive Bayes



Multinomial Naive Bayes

The classifier uses the frequencies of words in a document and estimates how likely those words are under each class.

3. Bayes' Rule

Bayes' theorem relates a posterior probability to a prior and a likelihood:

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

where:

- $P(c \mid d)$: probability of class c given document d
- $P(d \mid c)$: probability of observing document d in class c
- $P(c)$: prior probability of class c
- $P(d)$: probability of the document

For classification, $P(d)$ is the same for all classes.

Therefore:

$$P(c \mid d) \propto P(d \mid c)P(c)$$

4. Naive Bayes Assumption

Let:

$$d = (w_1, w_2, \dots, w_n)$$

be a document containing n words.

Naive Bayes assumes conditional independence between features given the class:

$$P(d \mid c) = \prod_{i=1}^n P(w_i \mid c)$$

Therefore:

$$P(c \mid d) \propto P(c) \prod_{i=1}^n P(w_i \mid c)$$

Why “Naive”?

The words in a document are not truly independent. The independence assumption is a simplifying approximation that makes the model computationally efficient.

5. Multinomial Naive Bayes

For text classification, the **Multinomial Naive Bayes** model uses word occurrence counts.
Let:

n_{wc} = number of occurrences of word w in class c

and:

$$N_c = \sum_{w \in V} n_{wc}$$

be the total number of word occurrences in class c .

Without smoothing:

$$P(w \mid c) = \frac{n_{wc}}{N_c}$$

The document score is:

$$P(c) \prod_{w \in d} P(w \mid c)^{n_{wd}}$$

where n_{wd} is the number of occurrences of w in document d .

6. Estimating the Class Prior

The prior probability of a class is estimated from the training documents.

If:

N = total number of training documents

and:

N_c = number of documents in class c

then:

$$P(c) = \frac{N_c}{N}$$

Example:

Class	Documents	Prior
Sports	400	0.40
Politics	300	0.30
Economy	200	0.20

7. Estimating Word Probabilities

Suppose the vocabulary is:

$$V = \{\text{team, goal, market, stock, match}\}$$

For the Sports class, suppose the word counts are:

Word	team	goal	market	stock	match
Count	20	15	0	0	10

The total number of word occurrences is:

$$N_{\text{Sports}} = 45$$

Without smoothing:

$$P(\text{market} \mid \text{Sports}) = \frac{0}{45} = 0$$

This zero causes an important problem.

8. The Zero-Probability Problem

Suppose a new document contains:

`d = team market match`

For Sports:

$$P(d \mid \text{Sports}) = P(\text{team} \mid S)P(\text{market} \mid S)P(\text{match} \mid S)$$

If:

$$P(\text{market} \mid S) = 0$$

then:

$$P(d \mid S) = 0$$

regardless of the other words.

Problem

One unseen word can force the entire document likelihood to zero.

9. Laplace Smoothing

For a vocabulary of size $|V|$, Laplace smoothing changes:

$$\frac{n_{wc}}{N_c}$$

to:

$$P(w \mid c) = \frac{n_{wc} + 1}{N_c + |V|}$$

The numerator is increased by one for every word.

The denominator must also increase because one count is added to every vocabulary item.

General Add- α Smoothing

$$P(w \mid c) = \frac{n_{wc} + \alpha}{N_c + \alpha|V|}$$

Laplace smoothing corresponds to:

$$\alpha = 1$$

10. Laplace Smoothing: Worked Example

Recall:

$$N_{\text{Sports}} = 45$$

and:

$$|V| = 5$$

For the unseen word market:

$$n_{\text{market},S} = 0$$

After Laplace smoothing:

$$P(\text{market} \mid S) = \frac{0 + 1}{45 + 5} = \frac{1}{50} = 0.02$$

For team:

$$P(\text{team} \mid S) = \frac{20 + 1}{45 + 5} = \frac{21}{50} = 0.42$$

For goal:

11. Why the Denominator Changes

Suppose:

$$V = \{a, b, c, d\}$$

and a class contains:

$$N_c = 10$$

word occurrences.

Before smoothing:

$$P(a \mid c) + P(b \mid c) + P(c \mid c) + P(d \mid c) = 1$$

With Laplace smoothing, one count is added to every word:

$$N_c + |V| = 10 + 4 = 14$$

Therefore:

$$\sum_{w \in V} \frac{n_{wc} + 1}{N_c + |V|} = \frac{N_c + |V|}{N_c + |V|} = 1$$

12. What if a Test Word Is Not in the Vocabulary?

Suppose the training vocabulary is:

$$V = \{\text{team}, \text{goal}, \text{match}\}$$

but the test document contains:

stadium

There are two common strategies.

Strategy 1: Ignore Unknown Words

Discard words that are not in the training vocabulary.

Strategy 2: Use an <UNK> Token

Include an unknown token in the vocabulary during training and map unseen test words to:

<UNK>

Practical point

13. Computing the Multiclass Score

For document d and class c :

$$P(c \mid d) \propto P(c) \prod_{w \in d} P(w \mid c)^{n_{wd}}$$

Direct multiplication can produce extremely small numbers.

Instead, use logarithms:

$$\log P(c \mid d) = \log P(c) + \sum_{w \in d} n_{wd} \log P(w \mid c) + K$$

where K is the same for every class and can be ignored.

Thus:

$$\text{score}(c, d) = \log P(c) + \sum_{w \in d} n_{wd} \log P(w \mid c)$$

14. Worked Multiclass Example

Consider three classes:

$$C = \{\text{Sports, Economy, Politics}\}$$

Suppose:

$$P(S) = 0.5, \quad P(E) = 0.3, \quad P(P) = 0.2$$

For a document:

$$d = \text{team market}$$

the classifier computes:

$$\text{score}(S) = \log P(S) + \log P(\text{team} \mid S) + \log P(\text{market} \mid S)$$

and similarly:

$$\text{score}(E)$$

$$\text{score}(P)$$

15. Multinomial Naive Bayes Algorithm

- 1 Build the vocabulary from the training set.
- 2 Count documents in each class.
- 3 Compute class priors:

$$P(c) = \frac{N_c}{N}$$

- 4 Count word occurrences for every class.
- 5 Apply Laplace smoothing:

$$P(w | c) = \frac{n_{wc} + 1}{N_c + |V|}$$

- 6 For each test document, compute the log score for every class.
- 7 Select the class with the largest score.

Count → Smooth → Log → Score → Predict

16. Core Data Structures

A simple implementation can maintain:

```
class_doc_count = {}  
word_count = {}  
total_word_count = {}
```

For example:

```
class_doc_count["sports"] = 400  
  
word_count["sports"]["goal"] = 120  
word_count["sports"]["team"] = 95  
  
total_word_count["sports"] = 5000
```

The vocabulary is shared:

```
vocabulary = {  
    "goal", "team", "match",  
    "market", "stock", ...  
}
```

17. Training the Classifier

```
from collections import Counter, defaultdict

class_doc_count = Counter()
word_count = defaultdict(Counter)
total_word_count = Counter()
vocabulary = set()

for document, label in train_data:

    class_doc_count[label] += 1

    words = tokenize(document)
    vocabulary.update(words)

    for word in words:
        word_count[label][word] += 1
        total_word_count[label] += 1
```

After processing the training set, the model has the statistics needed to estimate:

$$P(c) \quad \text{and} \quad P(w \mid c)$$

18. Laplace-Smoothing Function

```
def word_probability(word, label):  
    count = word_count[label][word]  
    total = total_word_count[label]  
    V = len(vocabulary)  
  
    return (count + 1) / (total + V)
```

The corresponding mathematical expression is:

$$P(w \mid c) = \frac{n_{wc} + 1}{N_c + |V|}$$

Notice that even when:

$$n_{wc} = 0$$

we obtain:

$$P(w \mid c) > 0$$

19. Prediction Function

```
import math

def predict(document):
    scores = {}

    words = tokenize(document)
    counts = Counter(words)

    N = sum(class_doc_count.values())

    for label in class_doc_count:

        prior = class_doc_count[label] / N
        score = math.log(prior)

        for word, count in counts.items():

            if word not in vocabulary:
                continue

            p = word_probability(word, label)
            score += count * math.log(p)

        scores[label] = score

    return max(scores, key=scores.get)
```

20. Why Use Log Probabilities?

Without logarithms:

$$P(c) \prod_{w \in d} P(w \mid c)^{n_{wd}}$$

may involve multiplying hundreds or thousands of numbers smaller than one.

For example:

$$0.1 \times 0.01 \times 0.001 \times \dots$$

can underflow numerically.

Using:

$$\log(ab) = \log a + \log b$$

gives:

$$\log P(c) + \sum_w n_{wd} \log P(w \mid c)$$

21. Evaluating a Multiclass Classifier

For C classes, construct a confusion matrix:

M_{ij} = number of documents with true class i predicted as j

For example:

	Sports	Economy	Politics
Sports	92	5	3
Economy	7	88	5
Politics	4	6	90

Useful metrics include:

- Accuracy
- Precision
- Recall
- Macro-F1
- Micro-F1

22. Naive Bayes: Advantages and Limitations

Advantages

- Simple probabilistic model.
- Very fast training.
- Fast prediction.
- Works well with sparse text features.
- Small memory footprint.
- Easy to implement and interpret.

Limitations

- Conditional independence assumption is simplistic.
- Ignores word order.
- Vocabulary and preprocessing affect performance.
- Rare words can produce unreliable estimates.
- Does not directly model semantic similarity.

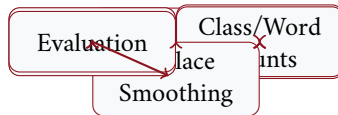
23. Assignment: SANAD Arabic Document Classification

Objective

Implement a **Multinomial Naive Bayes document classifier** for the SANAD Arabic news dataset.

- 1 Load the SANAD training, validation, and test data.
- 2 Apply Arabic text preprocessing.
- 3 Tokenize the documents.
- 4 Build a vocabulary from the training set.
- 5 Count documents and word frequencies for each class.
- 6 Estimate class priors.
- 7 Apply Laplace smoothing to word probabilities.
- 8 Implement classification using log probabilities.
- 9 Evaluate the classifier on the test set.

24. SANAD Assignment: Processing Pipeline



25. SANAD Assignment: Required Experiments

Students should investigate the effect of different feature representations.

Experiment	Representation	Purpose
1	Unigram counts	Basic baseline
2	Bigram counts	Add local context
3	Unigram + bigram	Combined features
4	No smoothing	Observe zero probabilities
5	Laplace smoothing	Handle zero counts
6	TF-IDF + NB	Compare feature weighting

Report

For each experiment, report accuracy, macro-F1, and a confusion matrix, and explain how smoothing and feature choices affect the results.

26. SANAD Assignment: Deliverables

Implementation

Submit a complete Python implementation of Multinomial Naive Bayes.

Required Components

- Data loading
- Arabic preprocessing
- Tokenization
- Vocabulary construction
- Class and word counting
- Laplace smoothing
- Log-probability calculation
- Multiclass prediction
- Evaluation

Analysis

27. Key Takeaways

- 1 Naive Bayes converts word statistics into class probabilities.
- 2 Multinomial Naive Bayes is particularly natural for count-based document features.
- 3 Bayes' rule provides the probabilistic foundation:

$$P(c \mid d) \propto P(c)P(d \mid c)$$

- 4 The naive assumption factorizes the document likelihood.
- 5 Laplace smoothing prevents zero word probabilities:

$$P(w \mid c) = \frac{n_{wc} + 1}{N_c + |V|}$$

- 6 Log probabilities make the implementation numerically stable.
- 7 SANAD provides a realistic Arabic document-classification problem for implementing the complete pipeline.

Any Questions?

Naive Bayes Multiclass Document Classification

Counts $\rightarrow$ Probabilities $\rightarrow$ Laplace Smoothing $\rightarrow$ Classification